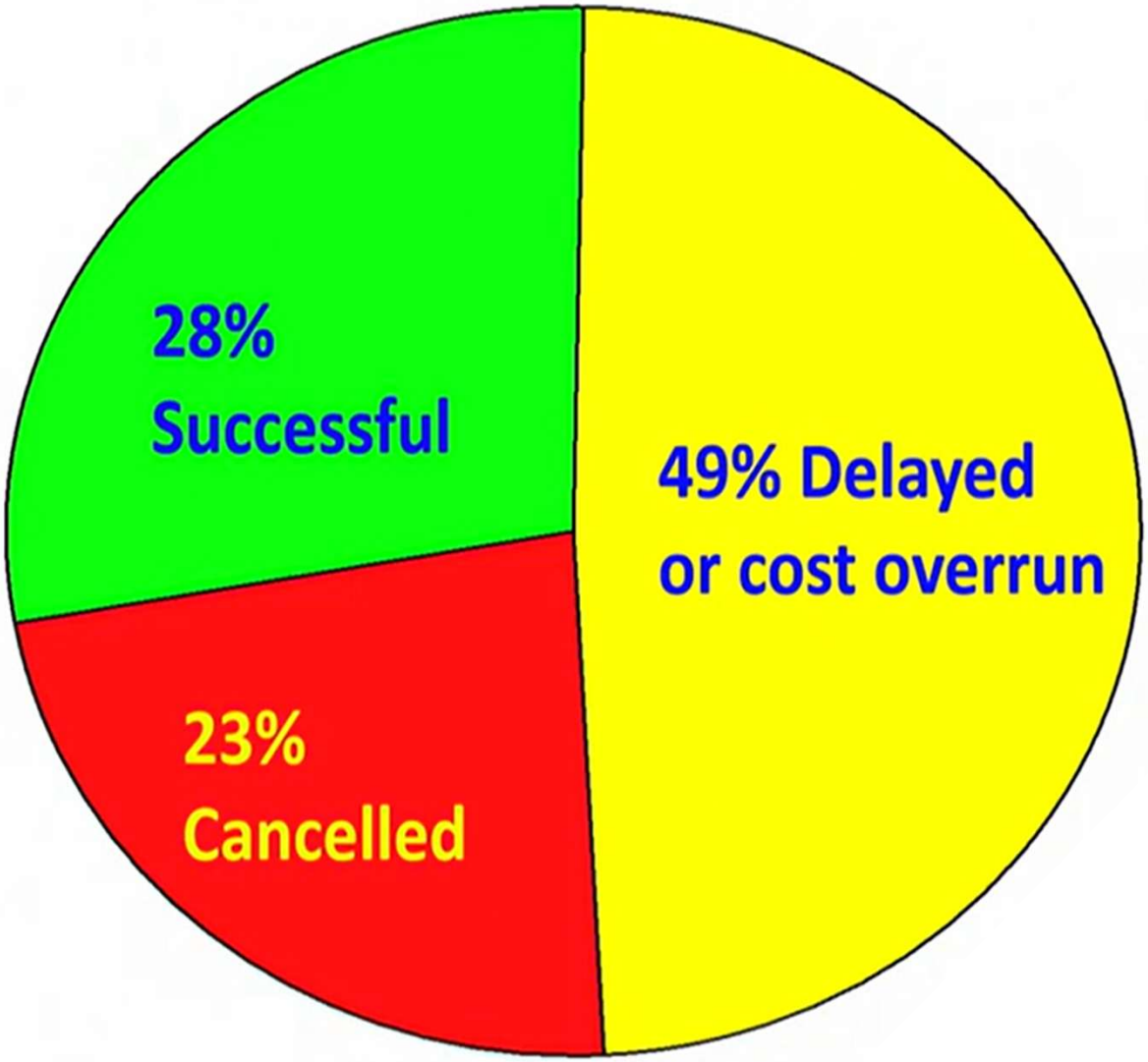
Software Project Management

Module 5

Software Project Management: Cost estimation- Project scheduling-Staffing-Software configuration management-Quality assurance-Software quality models-Project Monitoring-Risk management, etc. Emerging Trends in Software Engineering: Observing software engineering trends-Identifying "Soft trends"-Technology directions-Tools related trends

Organization of this Lecture:

- Introduction to Project Planning
- Software Cost Estimation
  - Cost Estimation Models
  - Software Size Metrics
  - Empirical Estimation
  - Heuristic Estimation
  - COCOMO
- Staffing Level Estimation
- Effect of Schedule Compression on Cost
- Summary


28%
Successful
49% Delayed
or cost overrun
23%
Cancelled

Software Crisis



- It is often the case that software products:
  - Fail to meet user requirements.
  - Expensive.
  - Difficult to alter, debug, and enhance.
  - Often delivered late.
  - Use resources non-optimally.

Which Factors are Contributing to the Software Crisis

- Larger problems
- Poor project management
- Lack of adequate training in software engineering
- Increasing skill shortage

In What Ways Management of Software Projects Differ from Management of Engineering Projects?

- Software is Intangible:
  - Software is invisible till it is ready and runs
- Large Change Impact:
  - A small change request can have terrible impact
  - Change is the rule rather than exception
- Intellectual work

Introduction

- Many software projects fail:
  - due to faulty project management practices:
    - It is important to learn different aspects of software project management.

Introduction

- Goal of software project management:
  - enable a group of engineers to work efficiently towards successful completion of a software project.

Responsibility of project managers

- Project proposal writing,
- Project cost estimation,
- Scheduling,
- Project staffing,
- Project monitoring and control,
- Software configuration management,
- Risk management,
- Managerial report writing and presentations, etc.

Introduction

- A project manager's activities are varied.
  - can be broadly classified into:
    - project planning,
    - project monitoring and control activities.

Project Planning

- Once a project is found to be feasible,
  - project managers undertake project planning.

Project Planning Activities

- Estimation:
  - Effort, cost, resource, and project duration
- Project scheduling:
- Staff organization:
  - staffing plans
- Risk handling:
  - identification, analysis, and abatement procedures
- Miscellaneous plans:
  - quality assurance plan, configuration management plan, etc.

Project planning

- Requires utmost care and attention --- commitments to unrealistic time and resource estimates result in:
  - irritating delays.
  - customer dissatisfaction
  - adverse affect on team morale
    - poor quality work
  - project failure.

Sliding Window Planning

- Involves project planning over several stages:
  - protects managers from making big commitments too early.
  - More information becomes available as project progresses.
    - Facilitates accurate planning

SPMP Document

- After planning is complete:
  - Document the plans:
  - in a Software Project Management Plan(SPMP) document.

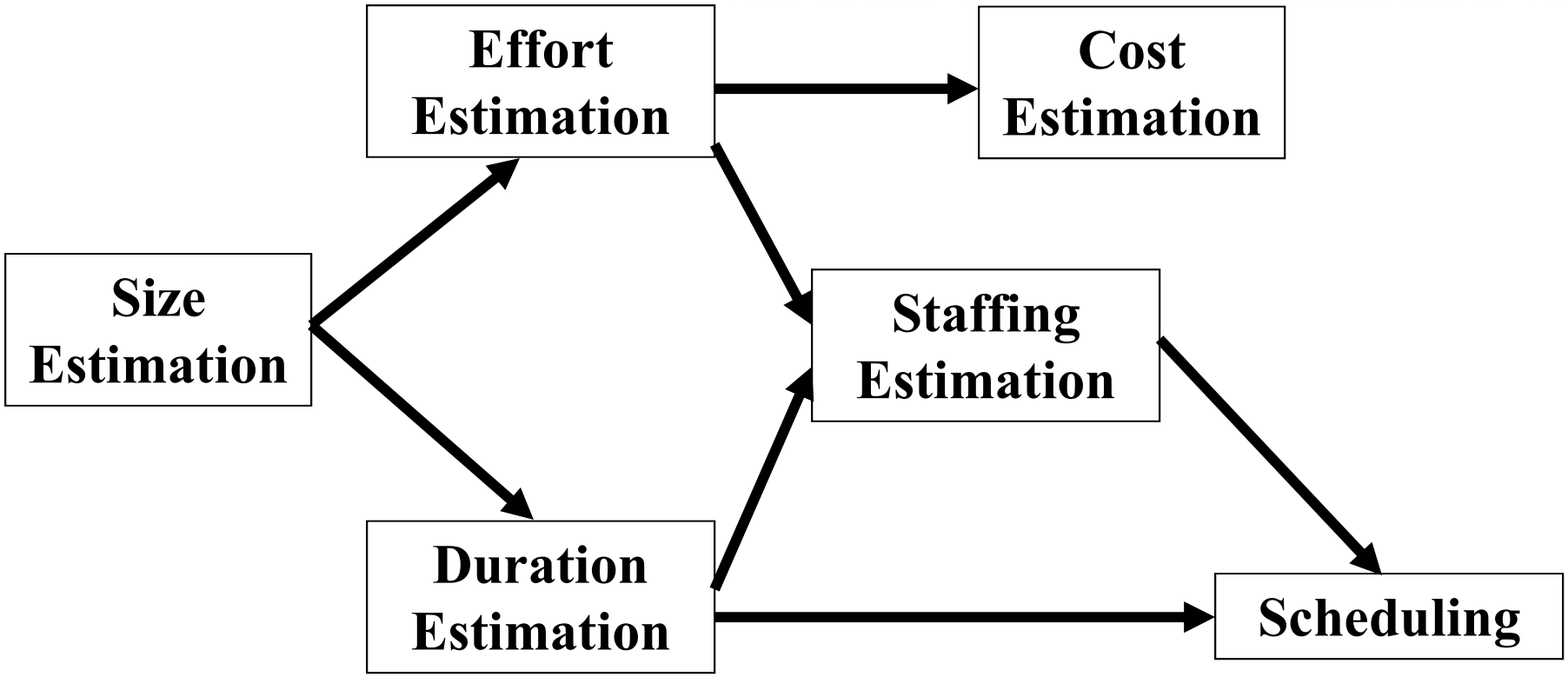
Organization of SPMP Document

- Introduction (Objectives,Major Functions,Performance Issues,Management and Technical Constraints)
- Project Estimates (Historical Data,Estimation Techniques,Effort, Cost, and Project Duration Estimates)
- Project Resources Plan (People,Hardware and Software,Special Resources)
- Schedules (Work Breakdown Structure,Task Network, Gantt Chart Representation,PERT Chart Representation)
- Risk Management Plan (Risk Analysis,Risk Identification,Risk Estimation, Abatement Procedures)
- Project Tracking and Control Plan
- Miscellaneous Plans(Process Tailoring,Quality Assurance)

Software Cost Estimation

- Determine size of the product.
- From the size estimate,
  - determine the effort needed.
- From the effort estimate,
  - determine project duration, and cost.

Software Cost Estimation



Software Cost Estimation

- Three main approaches to estimation:
  - Empirical
  - Heuristic
  - Analytical

Software Cost Estimation Techniques

- Empirical techniques:
  - an educated guess based on past experience.
- Heuristic techniques:
  - assume that the characteristics to be estimated can be expressed in terms of some mathematical expression.
- Analytical techniques:
  - derive the required results starting from certain simple assumptions.

Software Size Metrics

- LOC (Lines of Code):
  - Simplest and most widely used metric.
  - Comments and blank lines should not be counted.

Disadvantages of Using LOC

- Size can vary with coding style.
- Focuses on coding activity alone.
- Correlates poorly with quality and efficiency of code.
- Penalizes higher level programming languages, code reuse, etc.

Disadvantages of Using LOC
(cont...)

- Measures lexical/textual complexity only.
  - does not address the issues of structural or logical complexity.
- Difficult to estimate LOC from problem description.
  - So not useful for project planning

Function Point Metric

- Overcomes some of the shortcomings of the LOC metric
- Proposed by Albrecht in early 80's:
  - FP=4 #inputs + 5 #Outputs + 4 #inquiries + 10 #files + 10 #interfaces
- Input:
  - A set of related inputs is counted as one input.

Function Point Metric

- Output:
  - A set of related outputs is counted as one output.
- Inquiries:
  - Each user query type is counted.
- Files:
  - Files are logically related data and thus can be data structures or physical files.
- Interface:
  - Data transfer to other systems.

Function Point Metric (CONT.)

- Suffers from a major drawback:
  - the size of a function is considered to be independent of its complexity.
- Extend function point metric:
  - Feature Point metric:
  - considers an extra parameter:
    - Algorithm Complexity.

Empirical Estimation Techniques

- Expert Judgement:
  - An euphemism for guess made by an expert.
  - Suffers from individual bias.
- Delphi Estimation:
  - overcomes some of the problems of expert judgement.

Expert judgement

- Experts divide a software product into component units:
  - e.g. GUI, database module, data communication module, billing module, etc.
- Add up the guesses for each of the components.

Delphi Estimation:

- Team of Experts and a coordinator.
- Experts carry out estimation independently:
  - mention the rationale behind their estimation.
  - coordinator notes down any extraordinary rationale:
    - circulates among experts.

Delphi Estimation:

- Experts re-estimate.
- Experts never meet each other
  - to discuss their viewpoints.

Heuristic Estimation Techniques

- Single Variable Model:
  - Parameter to be Estimated=C1(Estimated Characteristic)^d1
- Multivariable Model:
  - Assumes that the parameter to be estimated depends on more than one characteristic.
  - Parameter to be Estimated=C1(Estimated Characteristic)^d1+ C2(Estimated Characteristic)^d2+…
  - Usually more accurate than single variable models.

PART A — INTRODUCTION

What is COCOMO?

- COCOMO (COnstructive COst MOdel) was proposed by Barry Boehm in 1981.
- It predicts effort and duration from the estimated software size and a number of cost drivers.

Size
in KLOC

→

Effort
in person-months

→

Duration
in months

Three COCOMO models

Basic
Size alone — a quick, approximate estimate

Intermediate
Size + 15 cost drivers refine the estimate

Detailed (Complete)
Each sub-system estimated separately, then summed

Development Modes

Organic

- Relatively small dev team.
- Well-understood application
- Familiar, stable environment
- e.g. Payroll Application, scientific data processing

Semi-detached

- Mix of experienced and inexperienced staff
- Between organic and embedded
- e.g. compilers, linkers, editors

Embedded

- Closely coupled with hardware or real-time systems
- Subject to strict operational and technical constraints
- Requires high reliability and precise timing. Great deal of innovation.
- May require specialized technical expertise
- E.g., Car braking control software, aircraft control systems, industrial controllers

Roughly: application programs · utility programs · system programs

Basic COCOMO Equations

EFFORT: $\text{Effort} = a\,(\text{KLOC})^{b}$ **person-months**

DEVELOPMENT TIME: $T_{dev} = c\,(\text{Effort})^{d}$ **months**

- KLOC — estimated size in thousands of lines of source code
- a, b, c, d — constants that depend on the mode
- Effort in person-months (PM); T_{dev} in months

ALSO USEFUL:
Average staff = Effort / T_{dev}
Productivity = KLOC / Effort

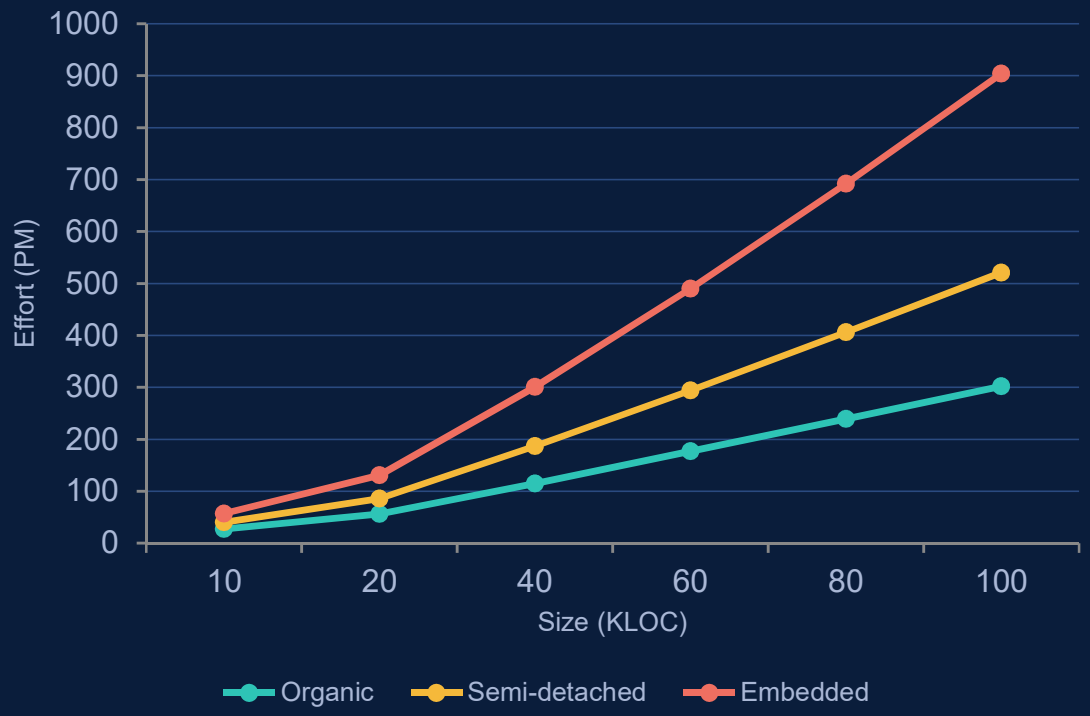
Constants for the Three Modes

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

$$\text{Effort} = a\,(\text{KLOC})^{b}\ \text{PM}$$

$$\text{Tdev} = c\,(\text{Effort})^{d}\ \text{months}$$

Effort vs Size



Effort is super-linear in size

The exponent is above 1 in every mode: 1.05, 1.12 and 1.20.

EXAMPLE (ORGANIC a_1=2.4, a_2=1.05)

32 KLOC → 91 PM

64 KLOC → 189 PM

Doubling the size a little more than doubles the effort.

Basic COCOMO Results for 30 KLOC and 60 KLOC

Effort and Development Time for the three modes

Mode	Size (KLOC)	Effort, E (person-months)	Development Time, T_{dev} (months)
Organic	30	85.35	13.5
	60	176.71	17.9
Semi-detached	30	135.36	13.9
	60	294.21	18.3
Embedded	30	213.23	13.9
	60	489.87	18.1

Effort equations (Basic COCOMO)

Organic: $E = 2.4\ (KLOC)^{1.05}$

Semi-detached: $E = 3.0\ (KLOC)^{1.12}$

Embedded: $E = 3.6\ (KLOC)^{1.20}$

Development time equation

$$T_{dev} = 2.5\ E^{d}$$

$d = 0.38$ (Organic)

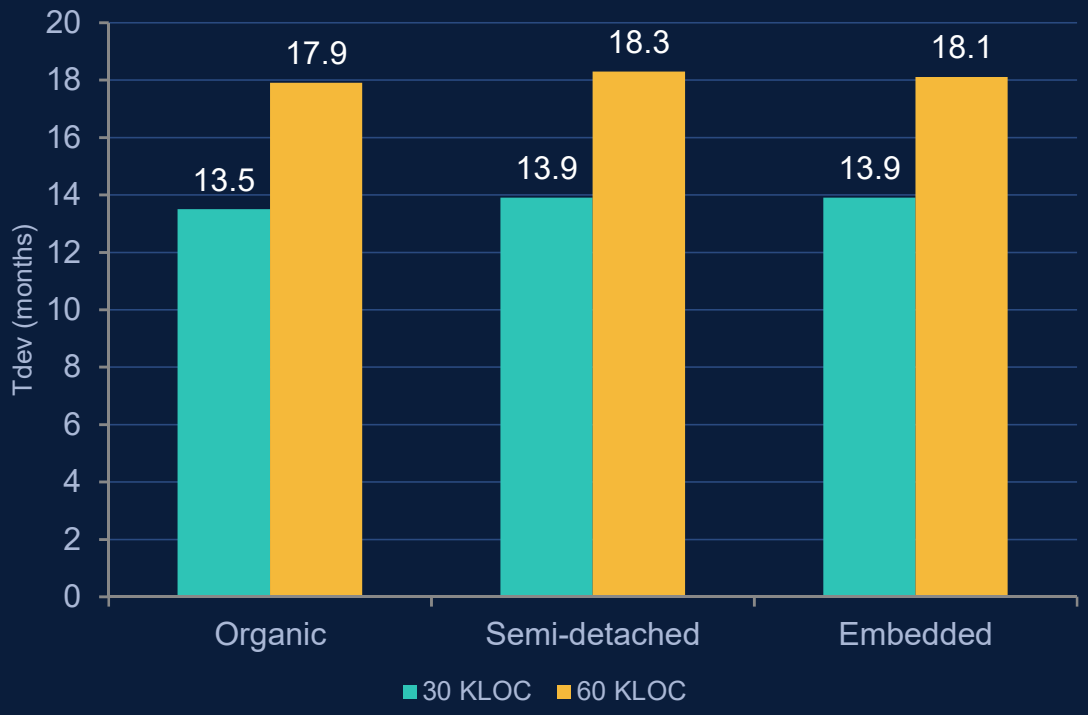
$d = 0.35$ (Semi-detached)

$d = 0.32$ (Embedded)

Key observation

- Development time is sub-linear in size.
- Doubling the size from 30 $KLOC$ to 60 KLOC increases T_{dev} from ≈ 14 months to ≈ 18 months for all three modes.

Development Time vs Size



Time is sub-linear in size
Doubling the size does not double the time.

About the same for all three modes: ≈ 14 months at 30 KLOC, ≈ 18 months at 60 KLOC.

Why? Larger products have more parallel activities that many engineers can carry out at once.

Example 1: 32 KLOC Organic Product

GIVEN An organic software product is estimated at 32,000 lines of source code (32 KLOC).

$\text{Effort} = 2.4 \times (32)^{1.05}$	= 91 PM
$\text{Tdev} = 2.5 \times (91)^{0.38}$	= 13.9 ≈ 14 months
Average staff = 91 / 14	≈ 6.5 persons

Effort ≈ 91 PM | Nominal development time ≈ 14 months

Example 2: 400 KLOC in All Three Modes

Size = 400 KLOC. Find effort and development time for each mode.

Mode	Effort	Effort (PM)	Tdev	Tdev (months)	Persons
Organic	$2.4 \times (400)^{1.05}$	1,295	$2.5 \times (1295)^{0.38}$	38.1	33.9
Semi-detached	$3.0 \times (400)^{1.12}$	2,463	$2.5 \times (2463)^{0.35}$	38.5	63.97
Embedded	$3.6 \times (400)^{1.2}$	4,773	$2.5 \times (4773)^{0.32}$	37.6	126.9

Embedded needs ≈ 3.7× the effort of organic, yet development time is almost the same (≈ 38 months).

Example 3: 200 KLOC Project

GIVEN: Size = 200 KLOC; average team experience on similar projects; schedule not very tight.

→ semi-detached mode.

Effort = $3.0 \times (200)^{1.12}$	= 1,133 PM
Tdev = $2.5 \times (1133)^{0.35}$	= 29.3 months
Average staff = Effort / Tdev = 1133 / 29.3	≈ 38.7 persons
Productivity = 200,000 LOC / 1133 PM	≈ 176.5 LOC/PM

1,133 PM | 29.3 months | ≈ 39 staff | 176.5 LOC/PM

PART C — INTERMEDIATE COCOMO

Why Intermediate COCOMO?

Basic COCOMO assumes effort and time depend on product size alone. In practice, other parameters also matter:

- **Reliability requirements**
- **CASE tools and modern facilities** (draw.io, PlantUML, Lucidchart, Pycharm, VS, Selenium, Jenkins)
- **Size of the data to be handled**

Intermediate COCOMO refines the Basic estimate using 15 cost drivers (multipliers).

Modern programming practices → initial estimate scaled DOWN

Stringent reliability requirements → initial estimate scaled UP

15 Cost Drivers in 4 Classes

Product (3)

- Required software reliability
- Size of application database
- Product complexity

Computer (4)

- Run-time performance constraints
- Memory constraints (how much RAM the software can use)
- Volatility of virtual machine (computing environment stability)
- Required turnaround time

Personnel (5)

- Analyst capability
- Software engineering capability
- Applications experience
- Virtual machine experience
- Programming language experience

Project (3)

- Use of software tools
- Application of SE methods
- Required development schedule

EAF and the Intermediate Equations

Effort Adjustment Factor (EAF) = product of the 15 cost driver multipliers

$$\text{Effort} = a\,(\text{KLOC})^{b} \times \text{EAF} \quad \text{Person-Months}$$

$$\text{Tdev} = c\,(\text{Effort})^{d} \quad \text{months}$$

Mode	a	b	c	d
Organic	3.2	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Multiplier < 1 lowers the effort, > 1 raises it.

Very high applications experience = 0.82

Very low programming experience = 1.14

Cost Driver Ratings (1 of 2)

Code	Cost driver	Very low	Low	Nominal	High	Very high	Extra high
Product attributes							
RELY	Required software reliability	0.75	0.88	1.00	1.15	1.40	–
DATA	Size of application database	–	0.94	1.00	1.08	1.16	–
CPLX	Product complexity	0.70	0.85	1.00	1.15	1.30	1.65
Computer attributes							
TIME	Run-time performance constraints	–	–	1.00	1.11	1.30	1.66
STOR	Memory constraints	–	–	1.00	1.06	1.21	1.56
VIRT	Volatility of virtual machine	–	0.87	1.00	1.15	1.30	–
TURN	Required turnaround time	–	0.87	1.00	1.07	1.15	–

Nominal = 1.00 (no effect). Above 1 raises the effort; below 1 lowers it. – = rating not defined.

Cost Driver Ratings (2 of 2)

Code	Cost driver	Very low	Low	Nominal	High	Very high	Extra high
Personnel attributes							
ACAP	Analyst capability	1.46	1.19	1.00	0.86	0.71	–
AEXP	Applications experience	1.29	1.13	1.00	0.91	0.82	–
PCAP	Software engineering capability	1.42	1.17	1.00	0.86	0.70	–
VEXP	Virtual machine experience	1.21	1.10	1.00	0.90	–	–
LEXP	Programming language experience	1.14	1.07	1.00	0.95	–	–
Project attributes							
MODP	Application of SE methods	1.24	1.10	1.00	0.91	0.82	–
TOOL	Use of software tools	1.24	1.10	1.00	0.91	0.83	–
SCED	Required development schedule	1.23	1.08	1.00	1.04	1.10	–

Gold = the two multipliers used in Example 4 (AEXP very high = 0.82, LEXP very low = 1.14).

Example 4: 300 KLOC Semi-detached

GIVEN 300 KLOC semi-detached project; developers have very high application experience (0.82) and very low programming-language experience (1.14).

EAF = 0.82 × 1.14 **= 0.9348**

Effort = $3.0 \times (300)^{1.12} \times 0.9348$ **≈ 1,668 PM**

Tdev = $2.5 \times (1668)^{0.35}$ **≈ 33.55 months**

Effort ≈ 1,668 PM | Scheduled time ≈ 33.55 months

Shortcoming of Basic and Intermediate

ASSUMPTION
Both models treat the software product as a single homogeneous entity.

→

REALITY

- Large systems are made of several smaller sub-systems
- Some sub-systems may be organic, others embedded
- Reliability requirements may be high for some of them
- We can estimate the effort for each subsystem separately, using Basic or Intermediate COCOMO depending on the level of detail required.

Solution: estimate each part separately → Detailed (Complete) COCOMO

PART D — DETAILED COCOMO

Detailed (Complete) COCOMO

1 Divide
Split the software into modules / sub-systems

→

2 Estimate
Apply COCOMO to each sub-system separately

→

3 Add
Sum the estimates of all sub-systems

→

4 Total
Overall cost of the whole system

Reduces the margin of error in the final estimate

Each module gets its own mode (organic / semi-detached / embedded) and its own cost-driver ratings.

PART D — DETAILED COCOMO

Example: MIS for a Multi-Office Organization

A Management Information System (MIS) for an organization with offices at several places across the country:

Database part
Semi-detached

GUI part
Organic

Communication part
Embedded

Cost of each component estimated separately → Summed to give the overall cost of the system

SUMMARY

The Three COCOMO Models

Model	Inputs	Key idea
Basic	KLOC only	Rough, quick estimate using mode constants
Intermediate	KLOC + 15 cost drivers (EAF)	Basic estimate refined by scaling with EAF
Detailed / Complete	Per sub-system KLOC + cost drivers	Estimate sub-systems separately and add; smaller error

An estimate is a prediction based on available information — not a guarantee.

Halstead's Software Science

- An analytical technique to estimate:
  - size,
  - development effort,
  - development time.

Halstead's Software Science

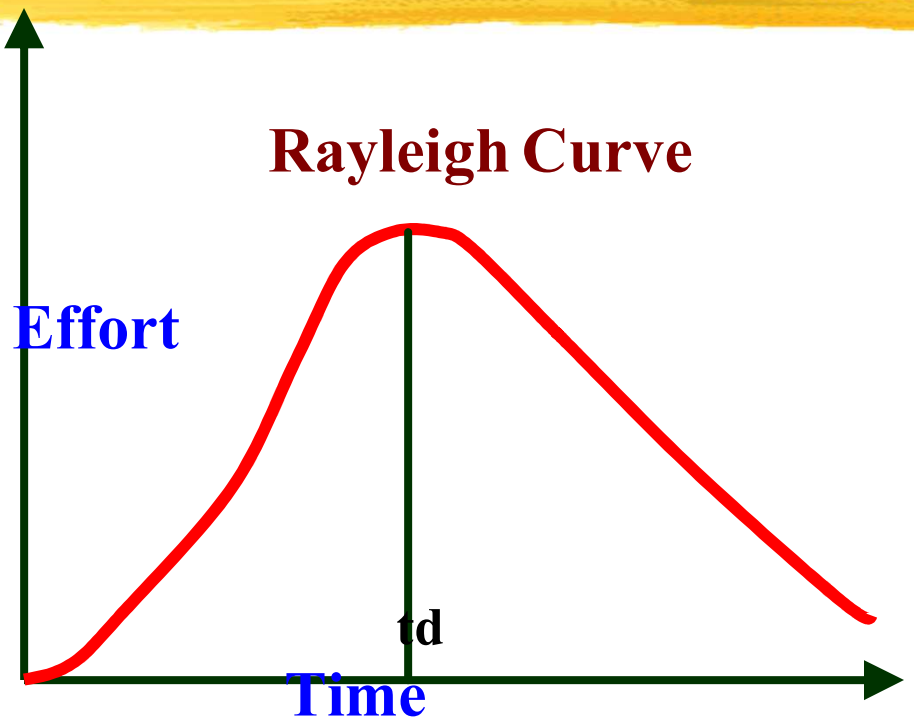
- Halstead used a few primitive program parameters
  - number of operators and operands
- Derived expressions for:
  - over all program length,
  - potential minimum volume
  - actual volume,
  - language level,
  - effort, and
  - development time.

Staffing Level Estimation

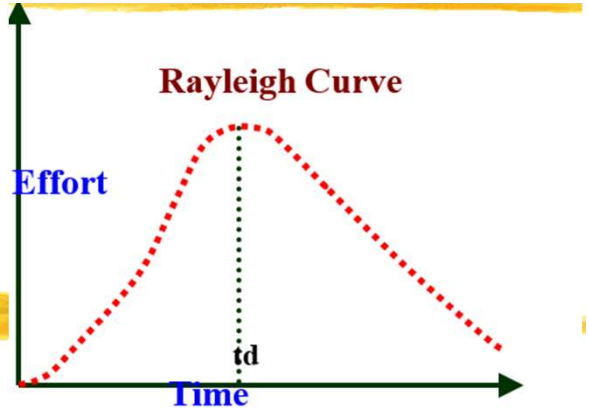
- Number of personnel required during any development project:
  - not constant.
- Norden in 1958 analyzed many R&D projects, and observed:
  - Rayleigh curve represents the number of full-time personnel required at any time.

Rayleigh Curve

- Rayleigh curve is specified by two parameters:
  - td the time at which the curve reaches its maximum
  - K the total area under the curve.
- L=f(K, td)

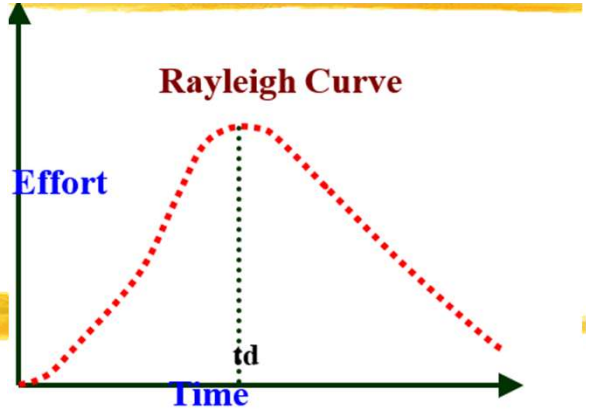


Rayleigh Curve



- Very small number of engineers are needed at the beginning of a project
  - carry out planning and specification.
- As the project progresses:
  - more detailed work is required,
  - number of engineers slowly increases and reaches a peak.

Rayleigh Curve



- the time at which the Rayleigh curve reaches its maximum value
  - corresponds to system testing and product release.
- After system testing,
  - the number of project staff falls till product installation and delivery.

Rayleigh Curve

- From the Rayleigh curve observe that:
  - approximately 40% of the area under the Rayleigh curve is to the left of td
  - and 60% to the right.

Putnam's Work:

- In 1976, Putnam studied the problem of staffing of software projects:
  - observed that the level of effort required in software development efforts has a similar envelope.
  - found that the Rayleigh-Norden curve
    - relates the number of delivered lines of code to effort and development time.

Putnam's Work (CONT.):

- Putnam analyzed a large number of army projects, and derived the expression:

$$L = C_k \times K^{1/3} \times t_d^{4/3}$$

- L is the effort expended and K is the size in KLOC.
  - td is the time to develop the software.
  - Ck is the state of technology constant
    - reflects factors that affect programmer productivity.

Putnam's Work (CONT.):

- Ck=2 for poor development environment
  - no methodology, poor documentation, and review, etc.
- Ck=8 for good software development environment
  - software engineering principles used
- Ck=11 for an excellent environment

Effect of Schedule Change on Cost

- Using the Putnam's expression for L,

$$K = \frac{L^3}{C_k^3 t_d^4} \qquad K = \frac{C_1}{t_d^4}$$

- For the same product size, C1=L3/Ck3 is a constant.

Effect of Schedule Change on Cost

- For two different schedules of the *same project type*:,

$$K_1 = \frac{C_1}{t_{d1}^4}, \quad K_2 = \frac{C_1}{t_{d2}^4}$$

$$\frac{K_1}{K_2} = \frac{t_{d2}^4}{t_{d1}^4}$$

$$K_1/K_2 = (t_{d2}/t_{d1})^4$$

Effect of Schedule Change on Cost (CONT.)

- Observe:
  - Effort (or cost) varies inversely with $td4$.
  - a relatively small compression in delivery schedule
    - can result in substantial penalty on human effort.

Example

If you reduce schedule time by **20%**:

$$\frac{t_{d2}}{t_{d1}} = 0.8$$

Then required effort changes roughly as:

$$\frac{K_1}{K_2} = (1/0.8)^4 = 2.44$$

- That means **effort (and cost) doubles** if you try to finish 20% faster!

Effect of Schedule Change on Cost (CONT.)

- Putnam model indicates extreme penalty for schedule compression
  - and extreme reward for expanding the schedule.
- Putnam estimation model works reasonably well for very large systems,
  - but seriously overestimates the effort for medium and small systems.

Effect of Schedule Change on Cost (CONT.)

- Boehm observed:
  - "There is a limit beyond which the schedule of a software project cannot be reduced by buying any more personnel or equipment."
  - This limit occurs roughly at 75% of the nominal time estimate.

Effect of Schedule Change on Cost (CONT.)

- If a project manager accepts a customer demand to compress the development time by more than 25%
  - very unlikely to succeed.
    - every project has only a limited amount of parallel activities
    - sequential activities cannot be speeded up by hiring any number of additional engineers.
    - many engineers have to sit idle.

Jensen Model

- Jensen model is very similar to Putnam model.
  - attempts to soften the effect of schedule compression on effort
  - makes it applicable to smaller and medium sized projects.

Jensen Model

- Jensen proposed the equation:

$$L = C_t \, e_t \, d \, K^{1/2}$$

  - Where,
    - Ct is the effective technology constant,
    - et is the effort adjustment factor for team capability, tools, etc.
    - d is the time to develop the software, and
    - K is the effort needed to develop the software.

Organization Structure

- Functional Organization:
  - Engineers are organized into functional groups, e.g.
    - specification, design, coding, testing, maintenance, etc.
  - Engineers from functional groups get assigned to different projects

Advantages of Functional Organization

- Specialization
- Ease of staffing
- Good documentation is produced
  - different phases are carried out by different teams of engineers.
- Helps identify errors earlier.

Project Organization

- Engineers get assigned to a project for the entire duration of the project
  - Same set of engineers carry out all the phases
- Advantages:
  - Engineers save time on learning details of every project.
  - Leads to job rotation

Team Structure

- Problems of different complexities and sizes require different team structures:
  - Chief-programmer team
  - Democratic team
  - Mixed organization

Democratic Teams

- Suitable for:
  - small projects requiring less than five or six engineers
  - research-oriented projects
- A manager provides administrative leadership:
  - at different times different members of the group provide technical leadership.

Democratic Teams

- Democratic organization provides
  - higher morale and job satisfaction to the engineers
  - therefore leads to less employee turnover.
- Suitable for less understood problems,
  - a group of engineers can invent better solutions than a single individual.

Democratic Teams

- Disadvantage:
  - team members may waste a lot time arguing about trivial points:
    - absence of any authority in the team.

Chief Programmer Team

- A senior engineer provides technical leadership:
  - partitions the task among the team members.
  - verifies and integrates the products developed by the members.

Chief Programmer Team

- Works well when
  - the task is well understood
    - also within the intellectual grasp of a single individual,
  - importance of early completion outweighs other factors
    - team morale, personal development, etc.

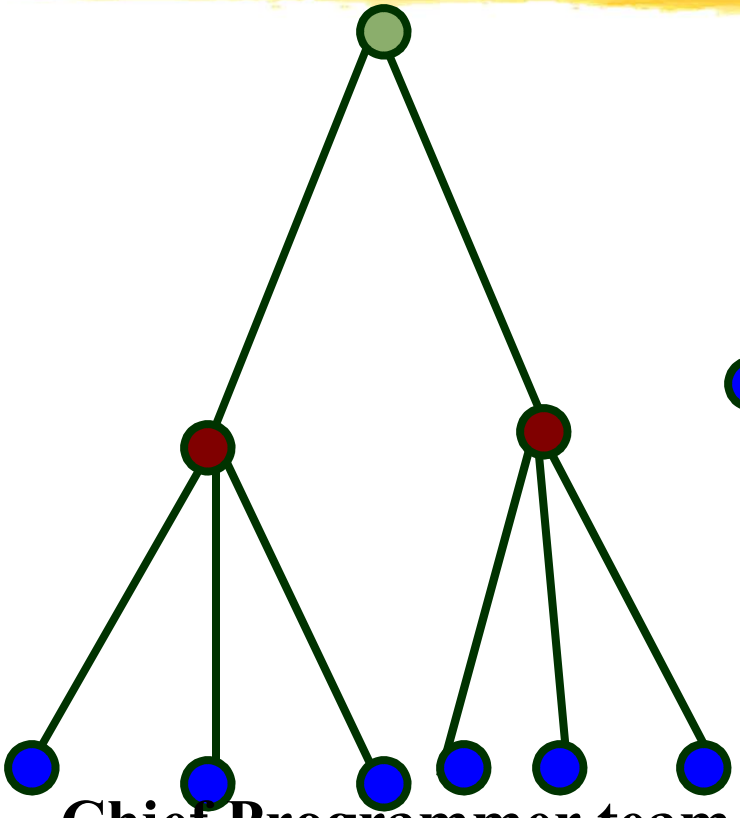
Chief Programmer Team

- Chief programmer team is subject to single point failure:
  - too much responsibility and authority is assigned to the chief programmer.

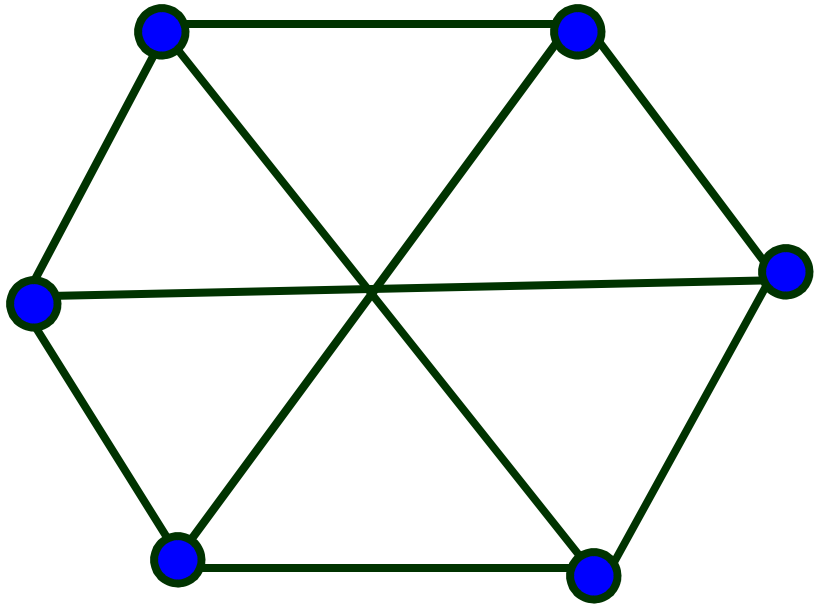
Mixed Control Team Organization

- Draws upon ideas from both:
  - democratic organization and
  - chief-programmer team organization.
- Communication is limited
  - to a small group that is most likely to benefit from it.
- Suitable for large organizations.

Team Organization

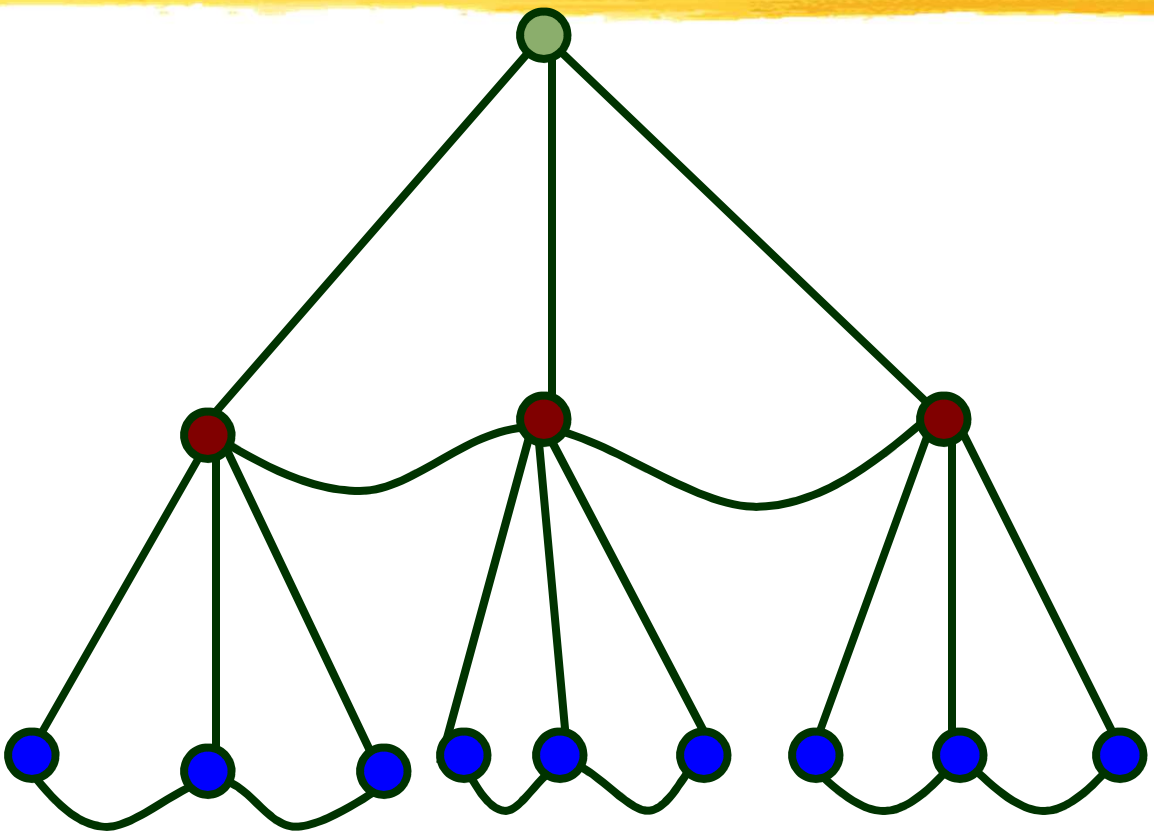


Chief Programmer team



Democratic Team

Mixed team organization



Summary

- We discussed the broad responsibilities of the project manager:
  - Project planning
  - Project Monitoring and Control

Summary

- To estimate software cost:
  - Determine size of the product.
  - Using size estimate,
    - determine effort needed.
  - From the effort estimate,
    - determine project duration, and cost.

Summary (CONT.)

- Cost estimation techniques:
  - Empirical Techniques
  - Heuristic Techniques
  - Analytical Techniques
- Empirical techniques:
  - based on systematic guesses by experts.
    - Expert Judgement
    - Delphi Estimation

Summary (CONT.)

- Heuristic techniques:
  - assume that characteristics of a software product can be modeled by a mathematical expression.
  - COCOMO
- Analytical techniques:
  - derive the estimates starting with some basic assumptions:
  - Halstead's Software Science

Summary (CONT.)

- The staffing level during the life cycle of a software product development:
  - follows Rayleigh curve
  - maximum number of engineers required during testing.

Summary (CONT.)

- Relationship between schedule change and effort:
  - highly nonlinear.
- Software organizations are usually organized in:
  - functional format
  - project format

Summary (CONT.)

- Project teams can be organized in following ways:
  - Chief programmer: suitable for routine work.
  - Democratic: Small teams doing R&D type work
  - Mixed: Large projects